

Task Manager Uygulaması Üzerinde Son Statik Analiz Raporu (Aşama 4)

Furkan Öztürk
Yazılım Mühendisliği
230229083

Taha Yasin Çiçek
Yazılım Mühendisliği
230229088

Betül Fikriye Turan
Yazılım Mühendisliği
220229073

Elmas Güneş
Yazılım Mühendisliği
220229054

Özet—Bu çalışma, Yazılım Kalite Güvencesi dönem projesinin dördüncü aşaması (*Son Statik Analiz*) kapsamında *Task Manager* web uygulaması üzerinde Aşama 2 ile birebir aynı araç, kural ve ayarlar kullanılarak gerçekleştirilen analizin sonuçlarını ve Aşama 2 ile karşılaştırmasını sunmaktadır. Analiz SonarQube 10.6 Community ve ESLint 8.57 ile yapılmış; Aşama 2’de raporlanan toplam 124 bulgudan (41 ESLint + 83 SonarQube) hiçbirisi son analizde gözlenmemiştir. `bugs=0`, `code_smells=0`, `vulnerabilities=0`, `security_hotspots=0`, `reliability_rating D`’den `A`’ya yükselmiş; teknik borç (`sqale_index`) 348 dakikadan 0 dakikaya inmiştir. Aşama 2 İlk Analiz Raporu’nda Aşama 4 için belirlenen tüm metrik hedefleri aşılarak karşılanmıştır.

Index Terms—yazılım kalite güvencesi, son statik analiz, SonarQube, ESLint, teknik borç, regresyon analizi, TypeScript, React

Tablo I
ÇALIŞMA ORTAMI VE ARAÇ VERSİYONLARI (AŞAMA 2 İLE ÖZDEŞ)

Bileşen	Versiyon
İşletim sistemi	Windows 11 Pro
Node.js	22.15.0
npm	10.9.2
Java (sonar-scanner için)	Eclipse Adoptium JDK 21.0.8
Docker Engine	28.3.2
ESLint	8.57.0
@typescript-eslint/parser	7.18.0
@typescript-eslint/eslint-plugin	7.18.0
eslint-plugin-react	7.35.0
eslint-plugin-react-hooks	4.6.2
SonarQube Community	10.6
sonar-scanner (npm)	3.1.0

I. GİRİŞ

Bu rapor, dönem projesinin dört aşamalı planının dördüncü ve son aşamasını belgelemektedir [1], [2]. Yönerge, Aşama 2 İlk Statik Analiz aşamasında kullanılan ile *birebir aynı* araç, kural ve ayarlarla son bir analiz yapılmasını ve İlk-Son karşılaştırmasının sunulmasını istemektedir. Bu çalışmada Aşama 3 Kod Güncelleme sürecinin somut sayısal etkisi, aynı analiz boru hattı çalıştırılarak ölçülmüştür.

II. YÖNTEM

A. Kullanılan Araçlar ve Konfigürasyon

Aşama 2 ile aynı bileşenler ve aynı konfigürasyon dosyaları kullanılmıştır. Versiyonlar Tablo I’de özetlenmiştir. `.eslintrc.cjs` ve `sonar-project.properties` dosyaları Aşama 2’den sonra Aşama 3 sürecinde değiştirilmemiş; yalnızca proje adı kozmetik olarak `task-manager` biçiminde sadeleştirilmiştir.

ESLint kural seti: `eslint:recommended` + `plugin:@typescript-eslint/recommended`; `client/src/` altında ek olarak `plugin:react/recommended`, `plugin:react-hooks/recommended` ve `plugin:react/jsx-runtime`. SonarQube kalite profili: yerleşik **Sonar way** (TypeScript).

B. Komutlar

Analiz aşağıdaki komutlarla, Aşama 2’deki `stage1` muadiline `stage2` etiketi kullanılarak yeniden üretilmiştir:

```
npm run lint:stage2
npm run sonar:up
sonar-scanner -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.token=$SONAR_TOKEN
node scripts/fetch-sonar-reports.mjs --stage stage2
```

III. BULGULAR (AŞAMA 4)

A. ESLint Sonuçları

Sonuç temizdir: 7 dosyada toplam 0 **error** ve 0 **warning** raporlanmıştır. Ham çıktı `asama_4/reports/stage2/eslint.json` dosyasındadır.

Tablo II
AŞAMA 4 ESLINT SONUÇLARI

Metrik	Değer
Sorunlu dosya sayısı	0
Toplam error	0
Toplam warning	0
Toplam bulgu	0

B. SonarQube Sonuçları

13 metrik Aşama 2’deki ile aynı API uçlarından çekilmiştir (`/api/measures/component` ve `/api/issues/search`). Sonuçlar Tablo III’de sunulmuştur. Severity dağılımı Tablo IV’de verilmiştir.

Tablo III
AŞAMA 4 SONARQUBE METRIKLERİ

Metrik	Değer
ncloc	5308
complexity	793
cognitive_complexity	2
bugs	0
code_smells	0
vulnerabilities	0
security_hotspots	0
duplicated_lines_density	%1.0
reliability_rating	1.0 (A)
sqale_rating	1.0 (A)
security_rating	1.0 (A)
sqale_index	0 dk
sqale_debt_ratio	%0.0

Tablo IV
AŞAMA 4 SONARQUBE SEVERITY DAĞILIMI

Tip	BLOCK.	CRIT.	MAJ.	MIN.	INFO	Top.
BUG	0	0	0	0	0	0
CODE_SMELL	0	0	0	0	0	0
Genel	0	0	0	0	0	0

IV. İLK (→) SON STATİK ANALİZ KARŞILAŞTIRMASI

A. ESLint Karşılaştırması

Tablo V ve Tablo VI, Aşama 2 ile Aşama 4 arasındaki ESLint farkını sunmaktadır.

Tablo V
ESLINT: İLK ANALİZ (AŞAMA 2) VS. SON ANALİZ (AŞAMA 4)

Metrik	Aşama 2	Aşama 4	Δ
Sorunlu dosya sayısı	7	0	-7
Toplam error	15	0	-15
Toplam warning	26	0	-26
Toplam bulgu	41	0	-41 (%100)

Tablo VI
ESLINT KURAL BAZINDA DEĞİŞİM

Kural	Aşama 2	Aşama 4	Δ
@typescript-eslint/no-explicit-any	36	0	-36
react/no-unescaped-entities	2	0	-2
@typescript-eslint/no-unused-vars	2	0	-2
react-hooks/exhaustive-deps	1	0	-1

B. SonarQube Karşılaştırması

Tablo VII 13 metriktaki değişimi, Tablo VIII severity dağılımındaki değişimi, Tablo IX ise en sık tetiklenen kuralların kural-bazlı değişimini göstermektedir.

C. Aşama 2 Hedefleri vs. Gerçekleşen

İlk Analiz Raporu'nda Aşama 4 için konulan tüm hedefler aşılıyor karşılanmıştır (Tablo X).

Tablo VII
SONARQUBE METRIKLERİ: İLK ANALİZ VS. SON ANALİZ

Metrik	Aşama 2	Aşama 4	Δ
ncloc	5153	5308	+155
complexity	776	793	+17
cognitive_complexity	2	2	0
bugs	14	0	-14 (%100)
code_smells	69	0	-69 (%100)
vulnerabilities	0	0	0
security_hotspots	2	0	-2 (%100)
Açık issue toplam	83	0	-83 (%100)
duplicated_lines_density	%1.3	%1.0	-%0.3
reliability_rating	D (4.0)	A (1.0)	↑ 3 seviye
sqale_rating	A	A	korundu
security_rating	A	A	korundu
sqale_index	348 dk	0 dk	-348
sqale_debt_ratio	%0.2	%0.0	-%0.2

Tablo VIII
SONARQUBE SEVERITY DAĞILIMI: İLK VS. SON

Tip × Severity	Aşama 2	Aşama 4	Δ
BUG × CRITICAL	1	0	-1
BUG × MINOR	13	0	-13
CODE_SMELL × CRITICAL	1	0	-1
CODE_SMELL × MAJOR	9	0	-9
CODE_SMELL × MINOR	59	0	-59
Toplam	83	0	-83

Tablo IX
SONARQUBE KURAL BAZINDA DEĞİŞİM (EN SIK 11 KURAL)

Kural	Aşama 2	Aşama 4	Δ
S6606 (?? vs.)	25	0	-25
S6853 (etiket bağı)	15	0	-15
S6848 (non-int. handler)	14	0	-14
S1082 (DOM event handler)	13	0	-13
S6759 (readonly prop)	6	0	-6
S3863 (duplicate import)	4	0	-4
S3358 (nested ternary)	2	0	-2
S2871 (sort comparator)	1	0	-1
S3776 (cognitive complexity)	1	0	-1
S3696 (literal throw)	1	0	-1
S3626 (redundant return)	1	0	-1

Tablo X
AŞAMA 2 HEDEFLERİ VS. AŞAMA 4 GERÇEKLEŞEN

Metrik	Aşama 2	Hedef	Aşama 4
ESLint error	15	0	0
ESLint warning	26	≤ 5	0
SonarQube BUG	14	≤ 2	0
SonarQube CODE_SMELL	69	≤ 20	0
BLOCKER+CRITICAL CODE_SMELL	1	0	0
duplicated_lines_density	%1.3	≤ %1.0	%1.0
reliability_rating	D	A	A
sqale_rating	A	A	A
security_rating	A	A	A
sqale_index	348 dk	≤ 120 dk	0 dk
security_hotspots	2	0	0

Aşama 2’de yalnızca *reviewed* statüsüne getirilmesi hedeflenen 2 güvenlik incelemesi noktası, Aşama 3’te gerçek kod düzeltmeleriyle (regex süper-lineer geri izleme için `isValidEmail()` saf fonksiyonu, framework version disclosure için `app.disable('x-powered-by')`) kapatılmıştır.

V. YORUM

Aşama 3 boyunca yapılan kod güncellemelerinin sayısal etkisi son analizle doğrulanmıştır. 41 ESLint bulgusu ve 83 SonarQube bulgusunun tamamı kapanmış; yalnızca yönergenin izin verdiği üç suppression hakkından biri (HTML5 native drag-and-drop için S6847) gerekçesi belgelenerek kullanılmıştır.

Toplam kod satırındaki +155’lik artış (`ncloc: 5153→5308`), Aşama 3’te eklenen yardımcı fonksiyonlardan (`parseProjectIdFilter`, `buildAssigneeIdsKey`, `describePatchChanges`, `isValidEmail`, `statusBadgeClass`), `ApiError` sınıfından, `TaskRow/TaskFormData/Id/ProjectMemberRow/SqliteError` tip tanımlarından ve erişilebilirlik refaktörünün CSS güncellemelerinden kaynaklanmaktadır. Bu artışa rağmen `cognitive_complexity` sabit (2) kalmış, `complexity` yalnızca yaklaşık %2.2 artmıştır — yani eklenen kod, karmaşıklık eklemekten mantığı daha küçük ve okunabilir parçalara bölmüştür.

VI. SONUÇ

İlk statik analiz (Aşama 2) ile son statik analiz (Aşama 4) arasındaki tüm metriklerde anlamlı iyileşme gözlenmiş, Aşama 2 İlk Analiz Raporu’nda Aşama 4 için konulan tüm hedefler aşılarak karşılanmıştır. Aşama 2’de raporlanan 124 bulgudan hiçbirisi Aşama 4 son analizinde gözlenmemiştir; açık güvenlik incelemesi noktaları gerçek düzeltmelerle ortadan kaldırılmış, `reliability_rating` D’den A’ya yükselmiş ve teknik borç (`sqale_index`) 0 dakikaya inmiştir.

KAYNAKLAR

- [1] SonarSource, “SonarQube Documentation,” <https://docs.sonarsource.com/sonarqube/>, Erişim: 2026-05-11.
- [2] ESLint, “ESLint – Pluggable JavaScript Linter,” <https://eslint.org/>, Erişim: 2026-05-11.
- [3] G. A. Campbell, “Cognitive Complexity: A New Way of Measuring Understandability,” SonarSource S.A., Teknik Rapor, 2018.
- [4] J.-L. Letouzey, “The SQALE Method for Evaluating Technical Debt,” 2012 Third International Workshop on Managing Technical Debt (MTD), IEEE, 2012, pp. 31–36, doi:10.1109/MTD.2012.6225997.
- [5] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed., Addison-Wesley Professional, 2018, ISBN:978-0134757599.
- [6] W. Cunningham, “The WyCash Portfolio Management System,” *ACM SIGPLAN OOPS Messenger*, vol. 4, no. 2, 1993, pp. 29–30, doi:10.1145/157710.157715.
- [7] ISO/IEC 25010:2011, “Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models,” International Organization for Standardization, Geneva, 2011.
- [8] N. Nagappan and T. Ball, “Static Analysis Tools as Early Indicators of Pre-Release Defect Density,” Proc. 27th International Conference on Software Engineering (ICSE), ACM/IEEE, 2005, pp. 580–586, doi:10.1145/1062455.1062558.

- [9] M. Tufano *et al.*, “When and Why Your Code Starts to Smell Bad,” Proc. 37th International Conference on Software Engineering (ICSE), IEEE, 2015, pp. 403–414, doi:10.1109/ICSE.2015.59.
- [10] OWASP Foundation, “OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks,” <https://owasp.org/Top10/>, Erişim: 2026-05-11.